

CRYPTOGRAPHIE ASYMÉTRIQUE ET OPENSSL

1. Contexte et Objectifs

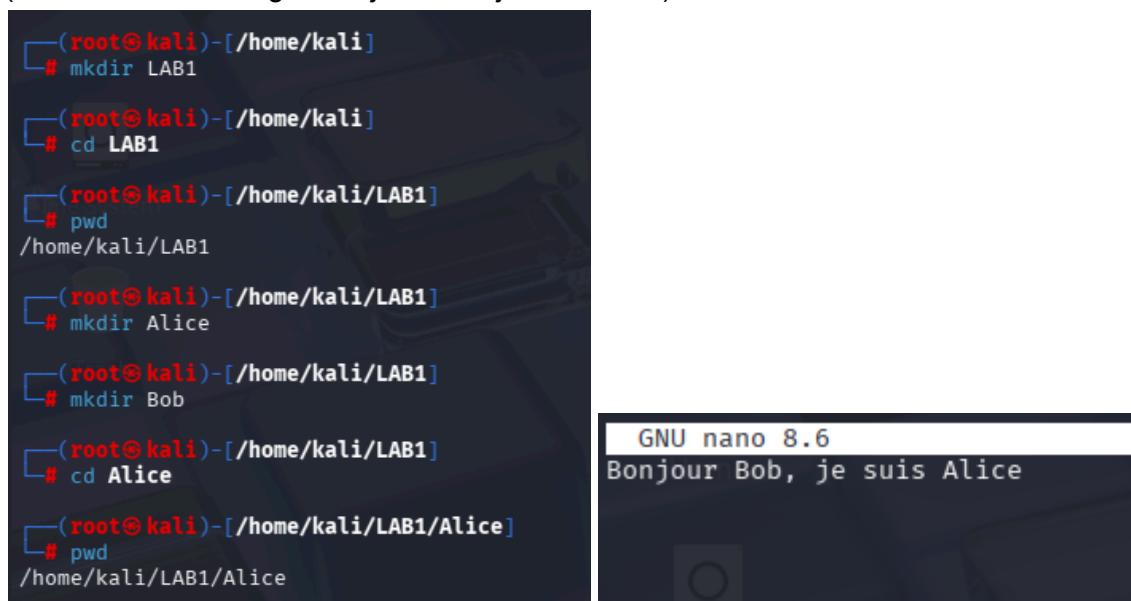
Ce laboratoire a pour objectif de manipuler la boîte à outils cryptographique **OpenSSL** sous environnement Linux (Kali). L'intervention vise à assimiler les concepts fondamentaux de la sécurité des échanges :

- La création de paires de clés (publique/privée).
- Le chiffrement et déchiffrement asymétrique de données.
- Les limites de performance du chiffrement asymétrique sur des volumes de données importants.
- La mise en œuvre de la signature électronique pour garantir l'authentification, l'intégrité et la non-répudiation.

2. Préparation de l'environnement et Génération des clés (Alice)

2.1. Création de l'arborescence et du document source

L'environnement de test a été préparé en créant un répertoire principal **LAB1**, contenant deux sous-répertoires distincts représentant nos deux entités : **Alice** et **Bob**. Dans le répertoire d'Alice, un fichier texte nommé **AliceDocument** a été créé à l'aide de l'éditeur **nano** (contenant le message "Bonjour Bob, je suis Alice").



```
(root@kali)-[/home/kali]
# mkdir LAB1

(root@kali)-[/home/kali]
# cd LAB1

(root@kali)-[/home/kali/LAB1]
# pwd
/home/kali/LAB1

(root@kali)-[/home/kali/LAB1]
# mkdir Alice

(root@kali)-[/home/kali/LAB1]
# mkdir Bob

(root@kali)-[/home/kali/LAB1]
# cd Alice

(root@kali)-[/home/kali/LAB1/Alice]
# pwd
/home/kali/LAB1/Alice
```

```
GNU nano 8.6
Bonjour Bob, je suis Alice
```

Figure 1 : Mise en place de l'environnement de travail et création du fichier source.

2.2. Génération de la paire de clés RSA

La sécurité asymétrique repose sur une paire de clés. Nous avons utilisé la commande `openssl genrsa -out AliceKeyPair` pour générer le trousseau initial d'Alice. L'option `-help` a préalablement permis d'explorer les paramètres de la commande.

```
(root@kali)-[/home/kali/LAB1/Alice]
# pwd
/home/kali/LAB1/Alice

(root@kali)-[/home/kali/LAB1/Alice]
# openssl genrsa -out AliceKeyPair

(root@kali)-[/home/kali/LAB1/Alice]
# ls
AliceDocument  AliceKeyPair
```

Figure 2 : Génération de la paire de clés RSA pour l'entité Alice.

2.3. Extraction et séparation des clés

Le fichier généré contenant à la fois la clé publique et la clé privée, il est impératif de les séparer. La clé publique a été extraite via la commande `openssl rsa -in AliceKeyPair -pubout -out AlicePublicKey`. Le fichier original, contenant la clé privée, a ensuite été renommé en `AlicePrivateKey` via la commande `mv` pour éviter toute confusion.

```
# ls
AliceDocument  AliceKeyPair

(root@kali)-[/home/kali/LAB1/Alice]
# openssl rsa -in AliceKeyPair -pubout -out AlicePublicKey
writing RSA key

(root@kali)-[/home/kali/LAB1/Alice]
# openssl rsa -help
Usage: rsa [options]

General options:
  -help          Display this summary

(root@kali)-[/home/kali/LAB1/Alice]
# ls
AliceDocument  AliceKeyPair  AlicePublicKey

(root@kali)-[/home/kali/LAB1/Alice]
# mv AliceKeyPair AlicePrivateKey
mv: cannot stat 'AliceKeyPair': No such file or

(root@kali)-[/home/kali/LAB1/Alice]
# mv AliceKeyPair AlicePrivateKey

(root@kali)-[/home/kali/LAB1/Alice]
# ls
AliceDocument  AlicePrivateKey  AlicePublicKey

(root@kali)-[/home/kali/LAB1/Alice]
#
```

Figure 3 : Séparation cryptographique et sécurisation de la clé privée.

```
GNU nano 8.6
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAqoxpskmun+X4UC2CVbJY
sxpvr98rJWF+C0cq2UUX2v5CavBlge42nZJIfiSxOY5++Q4DdVVC6atcNoOoRcVxt
26N/dAtPBZc0eszahc09lxeYpB836o4UtwqXhGLHqGF5dPKf7mXUSWXA920tXu2b
nQYYztUugPv8aM3SSi0kJdsrQSIEAyEEmg1Pt8sJY0xlrzbn9uPPCE5JvrjfSkVV
XWHaxHSVum5LsVG0IIixm02a7xtkGE2pQ4WQ6DEKtrb/JpHEt2+nzEndF1bNlFbJ
WzRXntlCgKtbemTeMILWwnZJ/c9c7xXJCsia9s2pB+n+cAu9qG2ih9TOi6PtEmPH
YQIDAQAB
-----END PUBLIC KEY-----
```

```
GNU nano 8.6
-----BEGIN PRIVATE KEY-----
MIIEuwwIBADANBgkqhkiG9w0BAQEFAASCBAUwggShAgEAAoIBAQCqjGmySa6f5fhQ
LYJVslizGm/3ysLYX4LRyrZRTHa/kJq8GWB7jadkhh+JLE5jn75DgN1VULpq1w2g
6hFxxG3bo390C08FLzR6zNqFzT2XF5ikHzfqjhS3CpeEYseoYXl08p/uZdRJJcd3
bS1e7ZudBhj01S6A+/xozdJKLSQL2ytBIgQDIQSaDU+3ywljTGWvNuf2488ITklw
uN9KRVVdYdrEdJW6bkuxUbQgiLGY7ZrvG2QYtalDhZDoMQq2tv8mkcS3b6fMSd0X
Vs2UVslbNFee2UKApnt6ZN4wgtZadkn9z1zvFckKyJr2zakH6f5wC72obaKH1M6L
o+0SY8dhAgMBAACGf8/CHuy0/epoWIpczEBLoc8EVHhx6aJXKFwRjutHVWkGbi0
07C1tTpxdX8pyQdjF4DhAyHbv/6/yuZ+lvMXKjiSToG3Xbrj2lhCkzhBu522xux9
EYQubKGZOF2/JX7QHvJRsy3Zikk0G7AH0htKun5m/d4k1/J+fdbIB45gSt/iYZl/
2Q2cdeFIkIQ2Dh2Czas7PHS+cF0Pdd89YnVihJsKK+VYz1EC3mqiKHB7eYYQvElo
1BSEGTK2HW04d1WanLueKnWmtx5axSGTcpx6Etya0/ntGS6EcGvdD9iUqtJgxWAC
Q2ArKsparhY71svjat5yJ7TCobKyYWL/mXuXs8ECgYEA1Wnbm911fSEWc41U13r+
o3720R5yQPTu8C87hx2YMFys8YUxdIJRqQDU1/uiPo9C0UZ3mCI9N/vJXAaEC9eb
iSiocapANXs7XaH1e0UKnHtkIVOT0U9sCfhLdN04vCRAoAvd73crrMZ8DtRhvxQL
duf2Nwt4KrR19y2L5TqXzZECgYEAzJTQHh+AFQouu2cw+TEUlsqsfE2/bEHa6LBJ
oDpEpFP3sIOQHtNm2DI9l7DWYqRFAhg/sCYaJh2tCGvGgJKntLZjvweWgCMkyzt
F0pZ0xrTA5jERVz/Q3/MubQGluj7kwBntObdo86u1Pgwn7YNc+yrXljx4hS5l4al
rWe7tNECgYBBQFqbl30qwyEgN4gzwlF2v+z8zehVIA6mdWqYB15u+wcN2ZGjc+0P
5Wcumoui4R6m6grLAz0AjwWKE4HwNP4jU8Ec79cuAv3FxS1PgW6bJccZwXoit0vr
SHWZx8H/FH6F8Nrsyt0ZuJ5Z0moIiFJxCr9nWSRmukKPTuFMMMVYQKBgAisysJs
PYcm3mJt9sKqVSAqc+u/dD2H8VtqlkSoE98P/HMsmUj1v57tSnVTN1QJeS+kjZb
Pls5E9eBkiY9MJ5whG8uu66/VTascgUwBs0P1yDQ6omybtSS58B8JMdja7+JBdzC
6WUXz4wSViEWpC1lPMMMBH1UuaX2PLJEss3xAoGBAK1X/yg/JMhA8x0Jd4oArAVs
gCGB0qf6xbi4PtK8Hdto3biEtSgwriGtsc/oR309DZjGqvWMCSQQMQ84ceWb43
BXsPzWXk+y1AYdFX7vsL74J3MklgIUCRUWoTLGbjiZVJZ1Eoz8Ay+LFh1NNTXV/B
UziaCzLMdwX8VNFud/uU
-----END PRIVATE KEY-----
```

3. Chiffrement et Déchiffrement Asymétrique

3.1. Préparation de l'entité Bob

Les mêmes opérations cryptographiques ont été reproduites dans le répertoire de Bob afin de lui générer sa propre paire de clés (**BobPrivateKey** et **BobPublicKey**).

```
(root@kali)-[/home/kali/LAB1/Bob]
# nano BobDocument

(rroot@kali)-[/home/kali/LAB1/Bob]
# openssl genrsa -out BobKeyPair

(rroot@kali)-[/home/kali/LAB1/Bob]
# ls
BobDocument  BobKeyPair

(rroot@kali)-[/home/kali/LAB1/Bob]
# openssl rsa -in BobKeyPair -pubout -out BobPublicKey
writing RSA key

(rroot@kali)-[/home/kali/LAB1/Bob]
# ls
BobDocument  BobKeyPair  BobPublicKey

(rroot@kali)-[/home/kali/LAB1/Bob]
# mv BobKeyPair BobPrivatePair

(rroot@kali)-[/home/kali/LAB1/Bob]
# ls
BobDocument  BobPrivatePair  BobPublicKey

(rroot@kali)-[/home/kali/LAB1/Bob]
# nano BobPrivatePair

(rroot@kali)-[/home/kali/LAB1/Bob]
# nano BobPublicKey

(rroot@kali)-[/home/kali/LAB1/Bob]
# mv BobKeyPair BobPrivateKey
mv: cannot stat 'BobKeyPair': No such file or directory

(rroot@kali)-[/home/kali/LAB1/Bob]
# mv BobPrivatePair BobPrivateKey

(rroot@kali)-[/home/kali/LAB1/Bob]
# ls
BobDocument  BobPrivateKey  BobPublicKey
```

Figure 4 : Création de l'identité cryptographique de Bob.

3.2. Chiffrement par Alice (Utilisation de la clé publique de Bob)

Pour envoyer un message confidentiel à Bob, Alice doit utiliser la **clé publique de Bob**.

1. La clé **BobPublicKey** a été copiée dans le répertoire d'Alice (**cp**).
2. Le fichier **AliceDocument** a été chiffré. *Note technique* : La commande **rsautl** étant dépréciée dans les versions récentes d'OpenSSL, la commande **pkeyutl** a été judicieusement utilisée : **openssl pkeyutl -encrypt -in AliceDocument -pubin -inkey BobPublicKey -out AliceDocumentEncrypted**.

```
(root@kali)~/home/kali/LAB1/Bob
# cp /home/kali/LAB1/Bob/BobPublicKey /home/kali/LAB1/Alice/

(root@kali)~/home/kali/LAB1/Bob
# cd ..

(root@kali)~/home/kali/LAB1
# cd Alice

(root@kali)~/home/kali/LAB1/Alice
# ls
AliceDocument  AlicePrivateKey  AlicePublicKey  BobPublicKey

(root@kali)~/home/kali/LAB1/Alice
# openssl rsautl -encrypt -in AliceDocument -pubin -inkey BobPublicKey -out AliceDocumentEncrypted
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.

(root@kali)~/home/kali/LAB1/Alice
# openssl pkeyutl -encrypt -in AliceDocument -pubin -inkey BobPublicKey -out AliceDocumentEncrypted

(root@kali)~/home/kali/LAB1/Alice
# ls
AliceDocument  AliceDocumentEncrypted  AlicePrivateKey  AlicePublicKey  BobPublicKey

(root@kali)~/home/kali/LAB1/Alice
#
```

Figure 5 : Chiffrement du document par Alice garantissant la confidentialité.

3.3. Déchiffrement par Bob (Utilisation de la clé privée de Bob)

Le fichier chiffré a été transféré dans le répertoire de Bob. Seul Bob, possédant la clé privée mathématiquement liée à la clé publique utilisée pour le chiffrement, peut lire le document. Le déchiffrement a été réalisé avec la commande `openssl pkeyutl -decrypt -in AliceDocumentEncrypted -inkey BobPrivateKey -out AliceDocumentDecrypted`. La lecture via `cat` confirme la récupération exacte du message original.

```
(root@kali)-[/home/kali/LAB1/Alice]
# cp /home/kali/LAB1/Alice/AliceDocumentEncrypted /home/kali/LAB1/Bob/

(root@kali)-[/home/kali/LAB1/Alice]
# cd ..

(root@kali)-[/home/kali/LAB1]
# cd Bob

(root@kali)-[/home/kali/LAB1/Bob]
# ls
AliceDocumentEncrypted  BobDocument  BobPrivateKey  BobPublicKey

(root@kali)-[/home/kali/LAB1/Bob]
# ls
AliceDocumentEncrypted  BobDocument  BobPrivateKey  BobPublicKey

(root@kali)-[/home/kali/LAB1/Bob]
# openssl pkeyutl -decrypt -in AliceDocumentEncrypted -inkey BobPrivateKey -out AliceDocumentDecrypted

(root@kali)-[/home/kali/LAB1/Bob]
# ls
AliceDocumentDecrypted  AliceDocumentEncrypted  BobDocument  BobPrivateKey  BobPublicKey

(root@kali)-[/home/kali/LAB1/Bob]
# cat AliceDocumentDecrypted
Bonjour Bob, je suis Alice

(root@kali)-[/home/kali/LAB1/Bob]
#
```

Figure 6 : Déchiffrement réussi par Bob.

4. Limites du Chiffrement Asymétrique

Pour démontrer les limitations techniques de l'algorithme RSA, un fichier volumineux a été généré aléatoirement par Alice via la commande `openssl rand -out LargeFile -base64 $((230 * 3/4))`. La tentative de chiffrement de ce fichier avec `BobPublicKey` a retourné une erreur : `data too large for key size`.

Conclusion technique : Le chiffrement asymétrique est très coûteux en ressources CPU et mathématiquement limité par la taille de la clé. Il n'est pas conçu pour chiffrer des données en masse, mais uniquement pour chiffrer de petites quantités d'informations (comme des clés de session pour un chiffrement symétrique).

```
96 openssl rand -out LargeFile -base64 $((230 * 3/4))
97 ls
98 openssl pkeyutl -encrypt -in LargeFile -pubin -inkey BobPublicKey -out LargeFileEncrypted
99 ls
100 cat LargeFileEncrypted
101 rm LargeFileEncrypted
102 clear
103 ls
104 openssl pkeyutl -encrypt -in LargeFile -pubin -inkey BobPublicKey -out LargeFileEncrypted

(root@kali)-[/home/kali/LAB1/Alice]
# openssl pkeyutl -encrypt -in LargeFile -pubin -inkey BobPublicKey -out LargeFileEncrypted
Public Key operation error
40C72AE2F97F0000:error:0200006E:rsa routines:ossl_rsa_padding_add_PKCS1_type_2_ex:data too large for key size:../cry
```

Figure 7 : Preuve matérielle de la limitation de taille en cryptographie asymétrique.

5. Signature Électronique et Vérification d'Intégrité

Cette étape valide la provenance d'un document (Authentification/Non-répudiation) et garantit qu'il n'a pas été altéré (Intégrité).

5.1. Hachage et Signature par Alice

Alice a créé un nouveau document `AuthData`.

1. **Hachage** : Une empreinte numérique (Hash) du fichier a été générée via l'algorithme SHA-256 : `openssl dgst -sha256 -out HashAuthData AuthData`.
2. **Signature** : Alice a ensuite chiffré ce hash avec sa **propre clé privée** en utilisant la commande `openssl rsautl -sign -in HashAuthData -inkey AlicePrivateKey -out AliceSignature`.


```

(root@kali)-[/home/kali/LAB1/Alice]
# nano AuthData

(root@kali)-[/home/kali/LAB1/Alice]
# ls
AliceDocument AliceDocumentEncrypted AlicePrivateKey AlicePublicKey AuthData BobPu

(root@kali)-[/home/kali/LAB1/Alice]
# cp /home/kali/LAB1/Alice/AlicePublicKey /home/kali/LAB1/Bob/

(root@kali)-[/home/kali/LAB1/Alice]
# openssl dgst -sha256 -out HashAuthData AuthData
dgst: Unknown option or message digest: sha256-out
dgst: Use -help for summary.
40B7638C1E7F0000:error:0308010C:digital envelope routines:inner_evp_generic_fetch:unsup

(root@kali)-[/home/kali/LAB1/Alice]
# dgst -help
Command 'dgst' not found, did you mean:
  command 'dgit' from deb dgit
  command 'dgtst' from deb ptscotch
  command 'dget' from deb devscripts
  command 'dist' from deb nmh
  command 'dust' from deb du-dust
Try: apt install <deb name>

(root@kali)-[/home/kali/LAB1/Alice]
# openssl dgst -sha256 -out HashAuthData AuthData

(root@kali)-[/home/kali/LAB1/Alice]
# ls
AliceDocument AliceDocumentEncrypted AlicePrivateKey AlicePublicKey AuthData BobPu

(root@kali)-[/home/kali/LAB1/Alice]
# cat HashAuthData
SHA2-256(AuthData)= 1609264b929f0754eca1915972291d17b76a56ddc8784ada532cfc7645c618cf

(root@kali)-[/home/kali/LAB1/Alice]
# openssl pkeyutl -sign -in HashAuthData -inkey AlicePrivateKey -out AliceSignature
Error: The non-raw input data length 85 is too long - max supported hashed size is 64

(root@kali)-[/home/kali/LAB1/Alice]
# openssl rsautl -sign -in HashAuthData -inkey AlicePrivateKey -out AliceSignature
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.

(root@kali)-[/home/kali/LAB1/Alice]
#

(root@kali)-[/home/kali/LAB1/Alice]
# ls
AliceDocument AliceDocumentEncrypted AlicePrivateKey AlicePublicKey AliceSignature

(root@kali)-[/home/kali/LAB1/Alice]
# cp /home/kali/LAB1/Alice/AliceSignature AuthData /home/kali/LAB1/Bob/

(root@kali)-[/home/kali/LAB1/Alice]
# cd ..

(root@kali)-[/home/kali/LAB1]
# cd Bob

(root@kali)-[/home/kali/LAB1/Bob]
# ls
AliceDocumentDecrypted AliceDocumentEncrypted AlicePublicKey AliceSignature AuthDa

```

Figure 8 : Création de la signature numérique par Alice.

5.2. Vérification par Bob

Le document source (**AuthData**), la signature (**AliceSignature**) et la clé publique d'Alice (**AlicePublicKey**) ont été transmis à Bob. Pour vérifier la validité de l'envoi, Bob exécute deux actions :

1. Il déchiffre la signature d'Alice à l'aide de sa clé publique pour récupérer le hash original : **openssl rsautl -verify -in AliceSignature -pubin -inkey AlicePublicKey -out HashAuthData**.
2. Il recalcule localement le hash du document reçu : **openssl dgst -sha256 -out HashBob AuthData**.

```
(root@kali)-[/home/kali/LAB1/Bob]
# openssl rsautl -verify -in AliceSignature -pubin -inkey AlicePublicKey -out HashAuthData
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.

(root@kali)-[/home/kali/LAB1/Bob]
# ls
AliceDocumentDecrypted AliceDocumentEncrypted AlicePublicKey AliceSignature AuthData BobDocument BobPrivateKey BobPublicKey HashAuthData

(root@kali)-[/home/kali/LAB1/Bob]
# openssl dgst -sha256 -out HashBob AuthData

(root@kali)-[/home/kali/LAB1/Bob]
# ls
AliceDocumentDecrypted AliceDocumentEncrypted AlicePublicKey AliceSignature AuthData BobDocument BobPrivateKey BobPublicKey HashAuthData HashBob
```

Figure 9 : Processus de vérification cryptographique côté destinataire.

5.3. Comparaison et Preuve d'Altération

Bob utilise la commande UNIX **diff HashBob HashAuthData** pour comparer les deux empreintes.

- **Cas nominal** : La commande ne retourne aucun résultat (null), prouvant que les hashes sont strictement identiques. L'intégrité et l'authenticité sont confirmées.
- **Cas d'altération** : Suite à une modification volontaire d'un des hashes, la commande **diff** met en surbrillance les différences de caractères, alertant immédiatement d'une corruption ou d'une attaque.

```
(root@kali)-[/home/kali/LAB1/Bob]
# diff HashBob HashAuthData

(root@kali)-[/home/kali/LAB1/Bob]
#

(root@kali)-[/home/kali/LAB1/Bob]
# diff HashBob HashAuthData
1c1
< SHA2-256(AuthData)= 2009264b929f0754eca1915972291d17b76a56ddc8784ada532cfc7645c618cf
> SHA2-256(AuthData)= 1609264b929f0754eca1915972291d17b76a56ddc8784ada532cfc7645c618cf
```

Figure 10 : Validation de l'intégrité par comparaison stricte des empreintes.

